databricks

Advance Lakeflow Jobs Features

LECTURE

Conditional and Iterative Tasks

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

We're now entering the realm of intelligent workflows that can make decisions and adapt their behavior based on runtime conditions. These aren't just linear sequences of tasks - they're dynamic workflows that can branch, loop, and make intelligent decisions based on data and processing results.

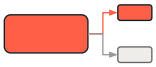
This capability transforms your workflows from simple automation to intelligent data processing systems.

Conditional and Iterative Tasks

Overview

Run if Conditional Task Dependencies

If Else Task



If/else

For Each Task

For each

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Three advanced task types enable sophisticated workflow patterns:

Run-if Conditional Task Dependencies: Control task execution based on the outcomes of upstream tasks, enabling workflows that can handle partial failures and complex dependency scenarios.

If/Else Tasks: Implement boolean conditional logic directly in your workflow, allowing branches based on data conditions, processing results, or business rules.

For Each Tasks: Enable iterative processing patterns where the same logic is applied to multiple data partitions or parameters, with configurable parallelism for performance optimization.

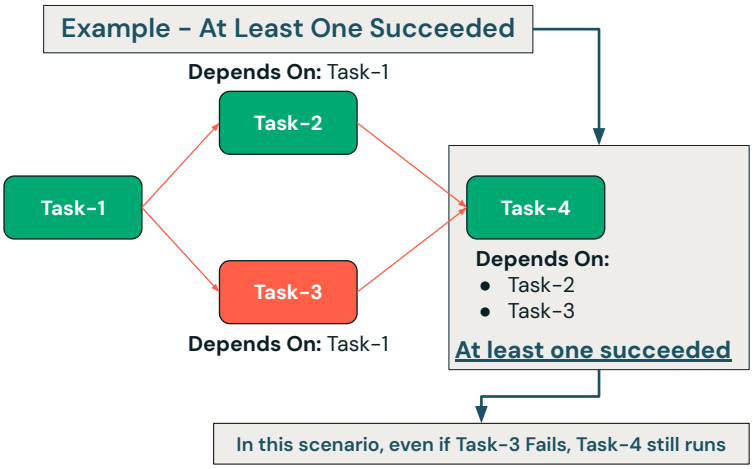
These task types can be combined to create sophisticated workflows that handle complex business logic while maintaining clarity and maintainability.

Conditional and Iterative Tasks

Run if Conditional Task Dependencies

Run if Conditional Task Dependencies

- Control task execution based on outcomes of **upstream tasks**
- Supports a variety of **dependency conditions** like:
  - All succeeded
  - At least one succeeded
  - None failed
  - And more



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Run-if conditional dependencies provide fine-grained control over task execution based on the outcomes of upstream tasks. You can define specific conditions that must be met for a task to run.
Available Dependency Conditions

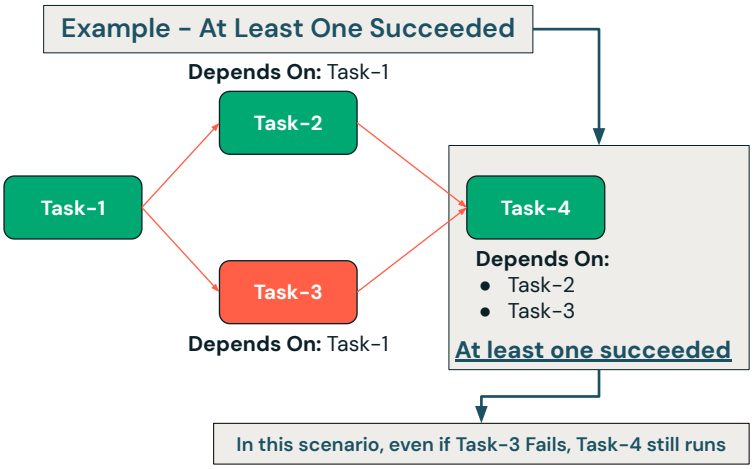
- All succeeded: This traditional dependency requires all upstream tasks to complete successfully before the next task can run.
- At least one succeeded: This condition is useful when you have redundant data sources or processing paths, allowing the workflow to proceed if even one of the required upstream tasks is successful.
- None failed: This allows a task to execute even if some upstream tasks were skipped, as long as no upstream tasks have explicitly failed.
- Custom combinations: This option enables the implementation of complex business logic that requires specific combinations of task outcomes.

Conditional and Iterative Tasks

Run if Conditional Task Dependencies

Run if Conditional Task Dependencies

- Control task execution based on outcomes of **upstream tasks**
- Supports a variety of **dependency conditions** like:
  - All succeeded
  - At least one succeeded
  - None failed
  - And more



© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Benefits of Conditional Dependencies

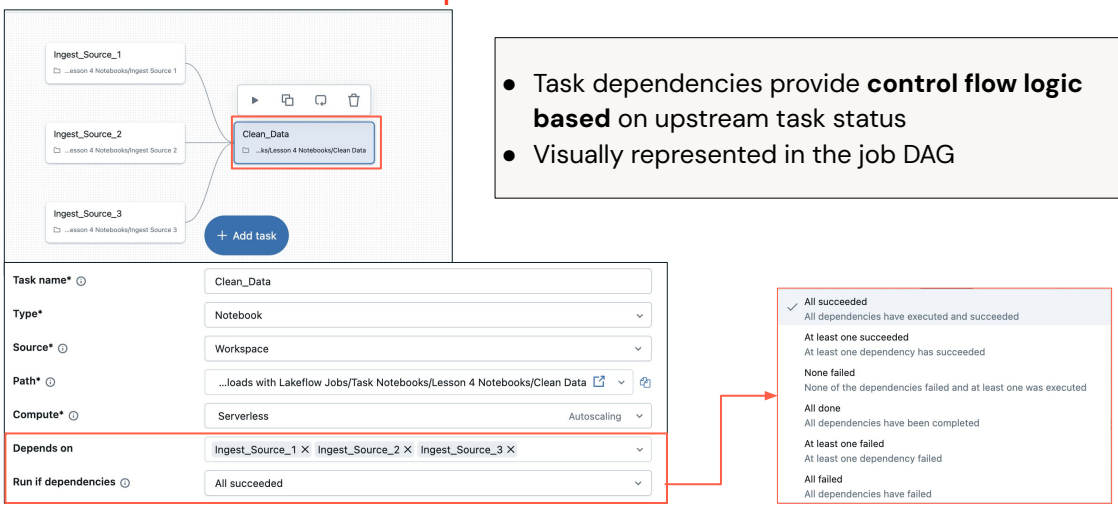
These dependencies enhance workflow resilience and enable sophisticated business logic. For instance, if `Task-4` is set to run when "at least one" of its predecessors (`Task-2` or `Task-3`) succeeds, it can still execute if `Task-3` fails while `Task-2` completes successfully. This prevents cascade failures and allows workflows to continue processing even when some components fail. It also helps mirror real-world business processes where multiple paths to success exist and partial failures do not halt the entire operation.

How to Configure

You can select these dependency conditions for a task within its configuration settings, under the “run-if dependencies” section.Which we are going to see next

Conditional and Iterative Tasks

Run if Conditional Task Dependencies



- Task dependencies provide **control flow logic based** on upstream task status
- Visually represented in the job DAG

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

The visual representation of conditional dependencies in the job DAG provides immediate understanding of workflow logic:

Dependency Visualization: Different line styles and colors indicate different dependency types, making complex logic easy to understand at a glance.

Troubleshooting Benefits: When failures occur, the visual representation immediately shows which tasks were affected and which could continue, accelerating root cause analysis.

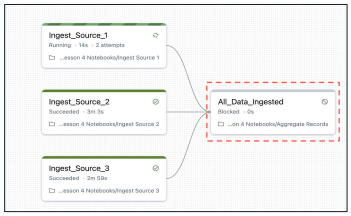
Team Communication: Visual workflows serve as living documentation that both technical and business stakeholders can understand, improving collaboration and change management.

Conditional and Iterative Tasks

Run if Conditional Task Dependencies

1. Job Initialization
The first three tasks are executed

2. Waiting for all task to complete (*All succeeded*)



3. Dependent Task Started

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

The execution flow diagram illustrates how conditional dependencies work in practice:

Job Initialization: The system evaluates all task dependencies and determines the initial execution set. Please note that all tasks are running concurrently.

Condition Evaluation: As tasks complete (successfully or with failures), the system continuously re-evaluates which downstream tasks can execute based on their dependency conditions. Please note that 'Ingest_Source_2' and 'Ingest_Source_3' have completed and are waiting for "Ingest_Source_1" to complete before proceeding to the 'All_Data_Ingested' task.

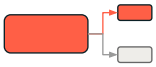
Dynamic Execution: This creates truly dynamic workflows where the execution path isn't predetermined but adapts based on actual processing results. Please note that once all three tasks are completed, the final task has started running.

Conditional and Iterative Tasks

If Else Task

Run if Conditional Task Dependencies

If Else Task



If/else

For Each Task

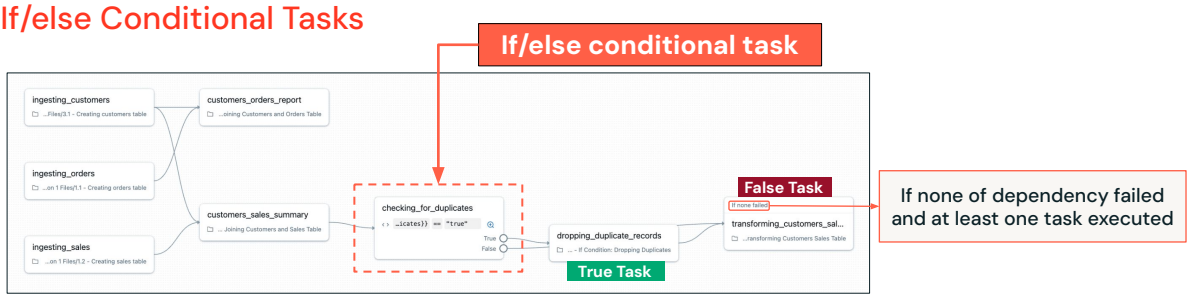
For each

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

if/Else tasks enable direct implementation of business logic within your workflows, moving beyond simple success/failure conditions to data-driven decision making.

Conditional and Iterative Tasks

If/else Conditional Tasks



If/else Conditional Tasks

- Adds boolean conditional logic to your workflow based on task results
- Allows branching based on specific conditions (e.g., data quality checks, record counts)
- Uses boolean operators (==, !=, >, >=, <, <=) to evaluate expressions

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

f/else conditional tasks add sophisticated boolean logic to workflows:

Condition Evaluation: Boolean operators (==, !=, >, >=, <, <=) evaluate expressions against task results, parameter values, or computed metrics.

Business Logic Examples:

- Data Quality Gates: Branch based on record counts, null percentages, or validation results
- Processing Volume Decisions: Use different processing strategies for large vs. small datasets
- Environment-Specific Logic: Execute different tasks based on environment parameters
- Business Rule Implementation: Implement complex business rules directly in workflow logic

Execution Requirements: The condition "If none of dependency failed and at least one task executed" ensures that conditional evaluation only occurs when meaningful upstream results are available.

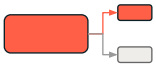
True/False Branches: Each branch can contain multiple tasks, enabling complex processing paths based on conditional outcomes.

Conditional and Iterative Tasks

For Each Task

Run if Conditional Task Dependencies

If Else Task



If/else

For Each Task

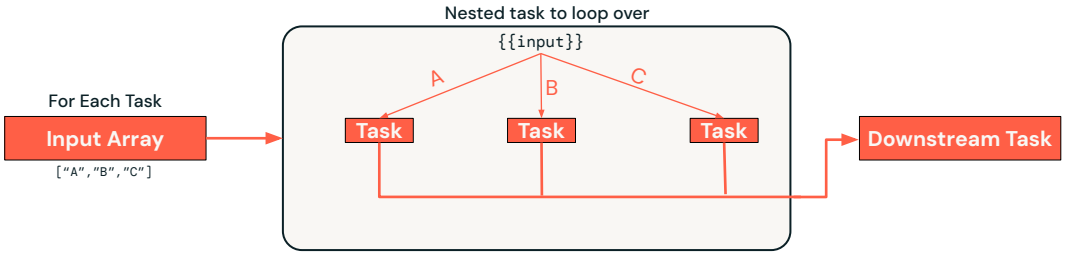
For each

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

For Each tasks enable powerful iterative processing patterns that maintain the benefits of visual workflow management while handling repetitive operations efficiently.

Conditional and Iterative Tasks

For Each Task



For Each Task

- For Each loops over an input array and runs the same nested task per item, passing it as `{{input}}`
- Iterations can run in parallel with configurable concurrency to speed up execution.
- Downstream dependencies attach to the For Each container, not to the nested task.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

For Each tasks provide sophisticated iteration capabilities:

Input Processing: The task loops over an input array, passing each item as `{{input}}` to the nested task. This creates clean, parameterized processing where the same logic handles different data partitions.

Parallel Execution: Configurable concurrency allows multiple iterations to run simultaneously, dramatically improving performance for independent processing tasks.

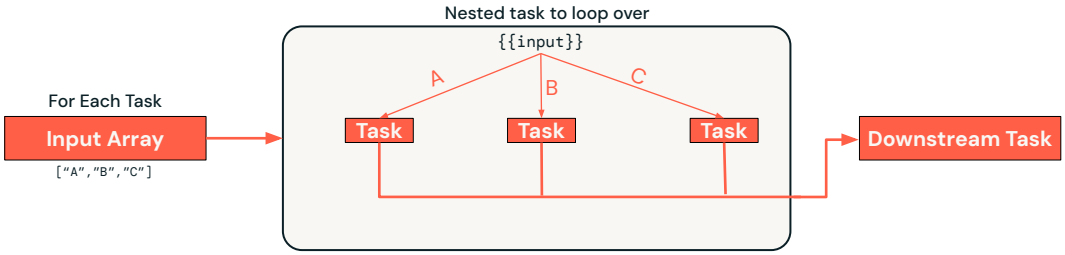
Dependency Management: Downstream tasks depend on the completion of the entire For Each container, not individual iterations. This simplifies dependency management while ensuring all iterations complete before downstream processing begins.

Use Case Examples:

- Geographic Processing: Process data for each state/region in parallel
- Time Period Processing: Handle different date ranges with the same logic
- Customer Segment Processing: Apply the same analysis to different customer segments
- File Processing: Process multiple files with identical logic

Conditional and Iterative Tasks

For Each Task



For Each Task

- For Each loops over an input array and runs the same nested task per item, passing it as `{{input}}`
- Iterations can run in parallel with configurable concurrency to speed up execution.
- Downstream dependencies attach to the For Each container, not to the nested task.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

The container concept is crucial for understanding For Each task behavior:

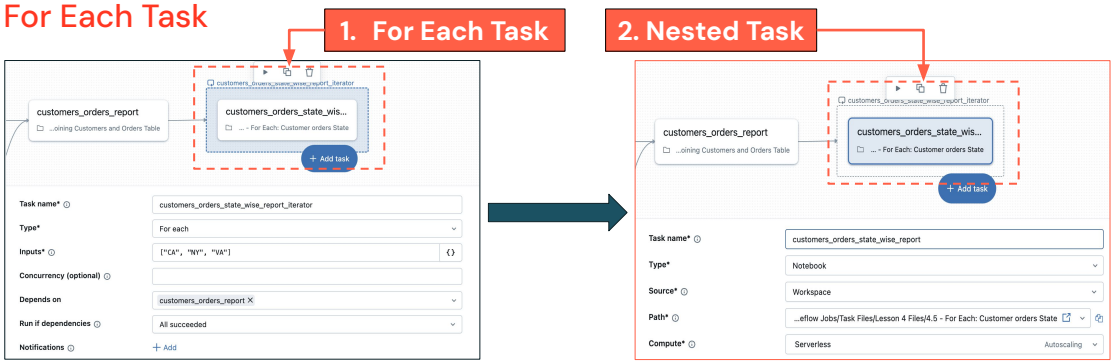
Container Management: The For Each task acts as a single logical unit in your workflow, even though it executes multiple iterations internally.

Dependency Simplification: Downstream tasks only need to depend on the For Each container, not each individual iteration, keeping workflow diagrams clean and manageable.

Resource Management: The container manages resource allocation across iterations, optimizing cluster utilization and preventing resource conflicts.

Conditional and Iterative Tasks

For Each Task



For Each task requires defining two components:

1. **The For Each task –** The top-level container task that manages the loop
2. **A nested task –** The actual task that executes for each iteration

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Implementing For Each tasks requires understanding two distinct components:

The For Each Container: This top-level task manages the iteration logic, input array processing, concurrency settings, and resource allocation. It defines how many iterations run in parallel and how the input array is processed.

The Nested Task: This is the actual work that gets performed for each iteration. It can be any task type - notebook, SQL, Python script, etc. The nested task receives each array item as `{{input}}` and processes it according to your business logic.

Configuration Flexibility: This separation allows you to configure iteration behavior independently from the processing logic, making For Each tasks both powerful and maintainable.

© Databricks 2025. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the Apache Software Foundation.

Thank you for completing this lesson and continuing your journey to develop your skills with us.